

Proyecto: Contador Distribuido de Frecuencia de Palabras con Ambassador y Circuit Breaker

Jimena Valentina Ortega Domínguez (318276), Diego Alonso López González (327969), Josué Jared Rueda Rivera (327944), Osvaldo Gutierrez Yepez Peña (327951)

Introducción

Los sistemas distribuidos constituyen en la actualidad la base de la gran mayoría de las aplicaciones modernas de alto rendimiento. Su capacidad para dividir el trabajo en nodos de cómputo permite escalar horizontalmente y procesar grandes volúmenes de datos que un único equipo no podría manejar de manera eficiente. Sin embargo, esta distribución introduce desafíos propios: fallos parciales, inconsistencias temporales y la necesidad de coordinar el trabajo entre nodos.

Este proyecto implementa un sistema distribuido de conteo de frecuencia de palabras que sirve como banco de pruebas para estudiar dos patrones y su eficacia en este tipo de entornos: el patrón de Ambassador y el patrón de Circuit breaker. Ambos patrones trabajan en capas diferentes de la arquitectura y se complementan entre sí para lograr un sistema que es observable y fácil de mantener.

El sistema está construido con Python y Flask, y distribuye el procesamiento de texto entre tres workers independientes coordinados mediante un nodo central. Esta arquitectura refleja los fundamentos del modelo MapReduce, donde cada worker procesa un fragmento del texto y el coordinador combina los resultados parciales.

Metodología

Arquitectura general:

El sistema se compone de cinco servicios que se ejecutan de forma independiente y se comunican mediante HTTP/REST:

- **Workers** : procesan fragmentos de texto y devuelven el conteo local de palabras. Cada uno escucha en su propio puerto (5001, 5002, 5003).
- **Ambassador**: actúa como intermediario entre el coordinador y los workers. Escucha en el puerto 6000, selecciona el worker disponible, gestiona reintentos y registra logs de cada solicitud.
- **Coordinador**: divide el texto de entrada en fragmentos equitativos, delega cada fragmento al Ambassador y combina los resultados parciales para producir el conteo global.

Patrón Ambassador:

El Ambassador es un proxy local que actúa en nombre del coordinador. Su función es desacoplar la lógica de comunicación (selección de workers, reintentos, logging) de la

lógica de negocio. Cuando el coordinador necesita procesar un fragmento, simplemente lo envía al Ambassador; este se encarga de elegir el worker más adecuado según la disponibilidad reportada por el Circuit Breaker.

Entre las responsabilidades concretas del Ambassador se encuentran:

- Balanceo de carga: selecciona al worker cuyo Circuit Breaker se encuentre en estado CLOSED, evitando enviar trabajo a nodos con fallos recientes.
- Reintentos automáticos: ante un fallo transitorio (timeout, error de conexión), reintenta la solicitud con otro worker sin que el coordinador tenga conocimiento del problema.
- Registro de trazas: cada solicitud queda registrada con su tiempo de respuesta, el worker seleccionado y el estado resultante, facilitando el diagnóstico.

Patrón Circuit Breaker:

El Circuit Breaker es un mecanismo de protección que evita que un worker en estado degradado siga recibiendo solicitudes que inevitablemente fallarán. Funciona como un interruptor automático con tres estados:

CLOSED	Operación normal	Permite el paso de todas las solicitudes
OPEN	3 fallos consecutivos	Bloquea solicitudes; el Ambassador redirige a otro worker
HALF -OPEN	10 segundo en estado OPEN	Permite una solicitud de pruebas; si tiene éxito, regresa a Closed

Flujo de Procesamiento:

El proceso completo sigue los siguientes pasos: el coordinador divide el texto de entrada en tres fragmentos de tamaño similar; envía cada fragmento al Ambassador de forma secuencial; el Ambassador consulta el estado de cada Circuit Breaker y selecciona un worker disponible; el worker procesa su fragmento y devuelve el conteo parcial; el Ambassador registra la respuesta y la reenvía al coordinador; finalmente, el coordinador fusiona los tres conteos parciales en el resultado global y lo compara con el conteo secuencial de referencia.

Resultados de la optimización

Worker 1:

```
○ (base) jimenaortegadominguez@MacBook-Air-de-Jimena project % python workers/worker1.py
[worker_1] Iniciando en puerto 5001...
* Serving Flask app 'worker1'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5001
* Running on http://192.168.1.73:5001
Press CTRL+C to quit
█
```

Worker 2:

```
○ (base) jimenaortegadominguez@MacBook-Air-de-Jimena project % python workers/worker2.py
[worker_2] Iniciando en puerto 5002...
* Serving Flask app 'worker2'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5002
* Running on http://192.168.1.73:5002
Press CTRL+C to quit
█
```

Worker 3:

```
○ (base) jimenaortegadominguez@MacBook-Air-de-Jimena project % python workers/worker3.py
[worker_3] Iniciando en puerto 5003...
* Serving Flask app 'worker3'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5003
* Running on http://192.168.1.73:5003
Press CTRL+C to quit
█
```

Ambassador:

```
○ (base) jimenaortegadominguez@MacBook-Air-de-Jimena project % python ambassador/ambassador.py
[Ambassador] Iniciando en puerto 6000...
[Ambassador] Workers configurados: ['worker_1', 'worker_2', 'worker_3']
[Ambassador] Timeout: 5.0s | Max reintentos: 3
* Serving Flask app 'ambassador'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:6000
* Running on http://192.168.1.73:6000
Press CTRL+C to quit
█
```

Coordinator:

```
(base) jimenaortegadominguez@MacBook-Air-de-Jimena project % python coordinator/coordinator.py
[Coordinador] Usando texto de prueba (265 palabras).

[Coordinador] INICIANDO CONTEO SECUENCIAL (Ground Truth)...

[Coordinador] Secuencial completado en 0.0002s | Palabras unicas: 142

[Coordinador] INICIANDO CONTEO DISTRIBUIDO...

[Coordinador] Texto dividido en 3 fragmentos:
  Fragmento #1: 88 palabras - 'Los sistemas distribuidos son un campo fundamental...'
  Fragmento #2: 88 palabras - 'son fundamentales en arquitecturas de microservici...'
  Fragmento #3: 89 palabras - 'que el sistema se recupere. El Circuit Breaker tie...'

[Coordinador] Enviando fragmento #1 (88 palabras) al Ambassador...
```

```
[Coordinador] Enviando fragmento #2 (88 palabras) al Ambassador...

[Coordinador] Enviando fragmento #3 (89 palabras) al Ambassador...
[Coordinador] Fragmento #3 procesado por worker_3
[Coordinador] Fragmento #2 procesado por worker_1
[Coordinador] Fragmento #1 procesado por worker_2

[Coordinador] Distribuido completado en 0.1789s | Palabras unicas: 142
```

RESULTADO DEL SISTEMA DISTRIBUIDO

Top 20 palabras más frecuentes (de 142 unicas):

1. de	15	
2. el	9	
3. en	8	
4. los	7	
5. sistemas	7	
6. un	7	
7. la	6	
8. es	5	
9. para	4	
10. distribuidos	4	
11. al	3	
12. breaker	3	
13. circuit	3	
14. del	3	
15. distribuido	3	
16. fallos	3	
17. que	3	
18. sistema	3	
19. texto	3	
20. como	3	

RESUMEN POR WORKER

```
worker_3: 89 palabras procesadas | 62 palabras unicas
worker_1: 88 palabras procesadas | 59 palabras unicas
worker_2: 88 palabras procesadas | 62 palabras unicas
```

COMPARACION DE RENDIMIENTO

```
Tiempo secuencial : 0.0002 segundos
Tiempo distribuido : 0.1789 segundos
Speedup           : 0.00x
Mejora            : -89466.5%
```

Consistencia de resultados: CONSISTENTE

Conclusión

El presente proyecto demuestra que Ambassador y Circuit Breaker son patrones complementarios que resuelven problemas distintos en sistemas distribuidos y cuya combinación produce una arquitectura robusta y mantenible.

El patrón Ambassador desacopla la complejidad de comunicación de la lógica de negocio. Al concentrar en un único componente la selección de workers, los reintentos y el logging, el coordinador queda libre de preocupaciones operativas y puede enfocarse exclusivamente en la partición del trabajo y la agregación de resultados. Este principio de separación de responsabilidades reduce el acoplamiento, facilita las pruebas unitarias y permite modificar la estrategia de comunicación (por ejemplo, cambiar el algoritmo de balanceo) sin tocar el código del coordinador.

El patrón Circuit Breaker aporta resiliencia frente a fallos parciales, uno de los escenarios más frecuentes y difíciles de gestionar en sistemas distribuidos. Al cortar rápidamente las solicitudes hacia un nodo degradado, evita el efecto cascado donde un worker lento o caído termina bloqueando todo el sistema al acumular solicitudes en espera. El mecanismo de autorrecuperación (estado HALF-OPEN) garantiza que el sistema vuelva a su capacidad plena tan pronto como el worker se restaure, sin requerir reinicio manual ni intervención del operador.

En conjunto, estos dos patrones ilustran un principio fundamental del diseño de sistemas distribuidos modernos: la fiabilidad no se logra eliminando los fallos, sino diseñando el sistema para tolerarlos y recuperarse de ellos de forma autónoma.